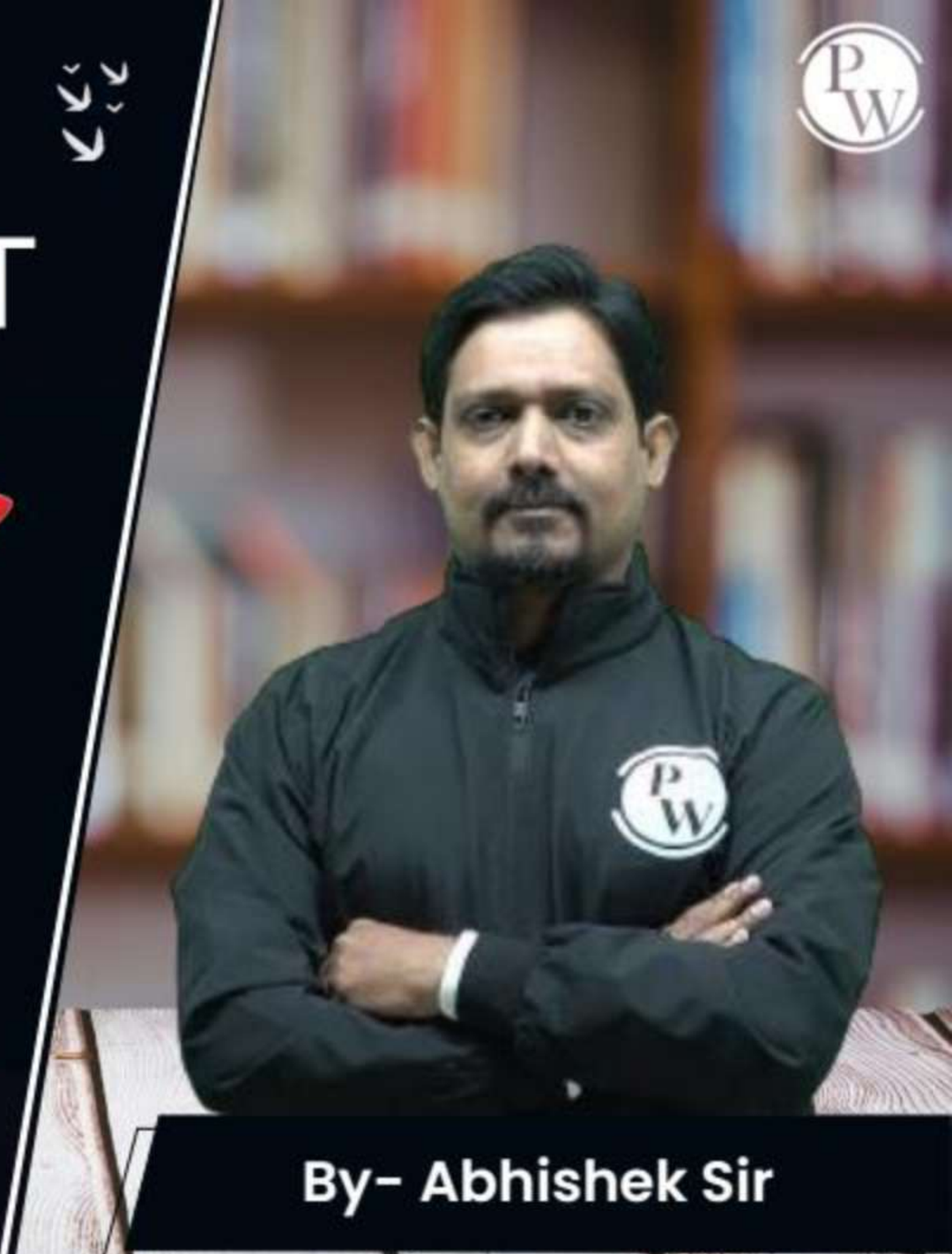
Computer Science & IT



C programming

Function & Storage Class

Lecture No. 05



By- Abhishek Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

Nested Recursion

Indirect Recursion

Slide

Topics to be Covered



Topic — Tower of Hanoi

Topic — Counting function Call

Topic — Computing value of Recursive function

Topic — Exteon

Topic

Slide



Tower of Hanoi

Classical problem

1. There are n disc is given of different size
2. 3 towers are given marked as L, M and R
3. No large disc will top of smaller disc.
4. We need to transfer all n disc from tower L to tower R using tower M
5. Only 1 disc can be moved at a time

Slide



Tower of Hanoi

Same

TOH(n, L, M, R) Destination

Temp

we are required to move n disc from tower L to tower R using tower M



Tower of Hanoi

TOH(n, L, M, R)

TOH(n-1, L, R, M)

Bottom of R which disc will be present





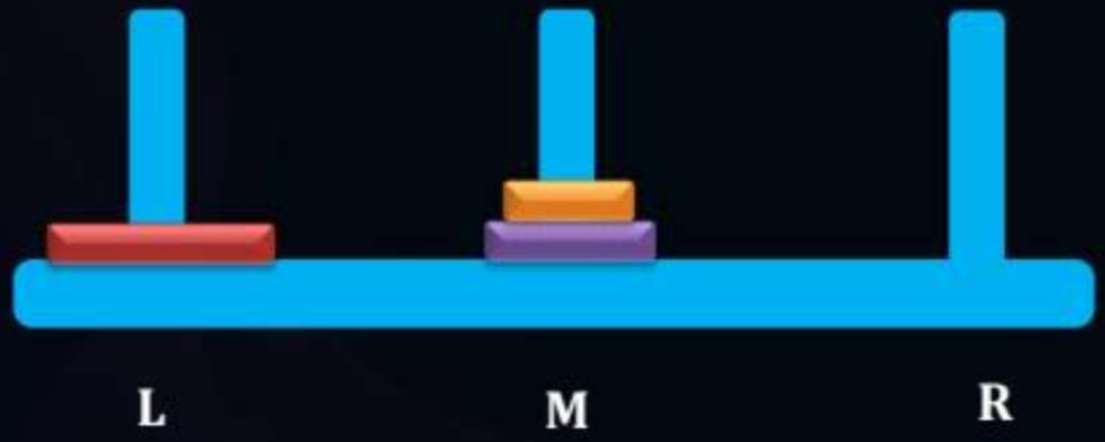
Tower of Hanoi





Tower of Hanoi

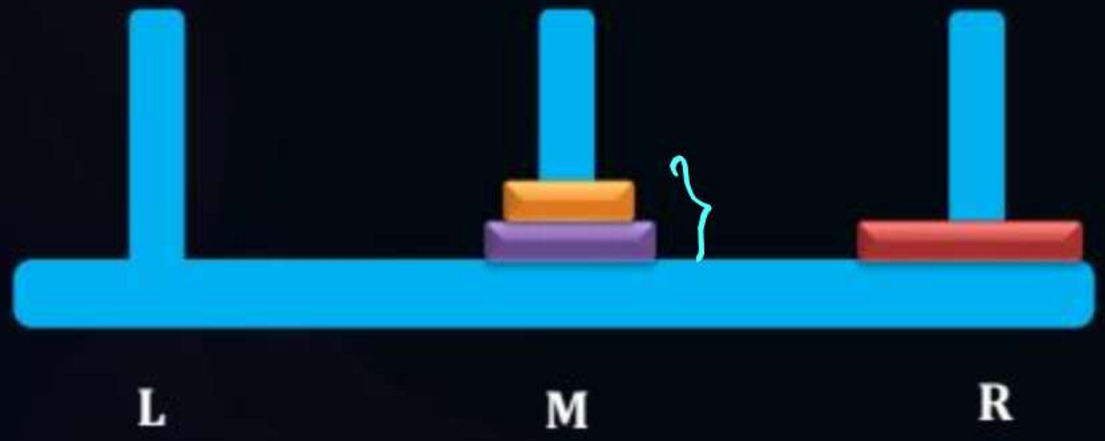
Move n^{th} disc from L → R





Tower of Hanoi

TOH(n-1, M, L, R)





Tower of Hanoi





Tower of Hanoi





Tower of Hanoi

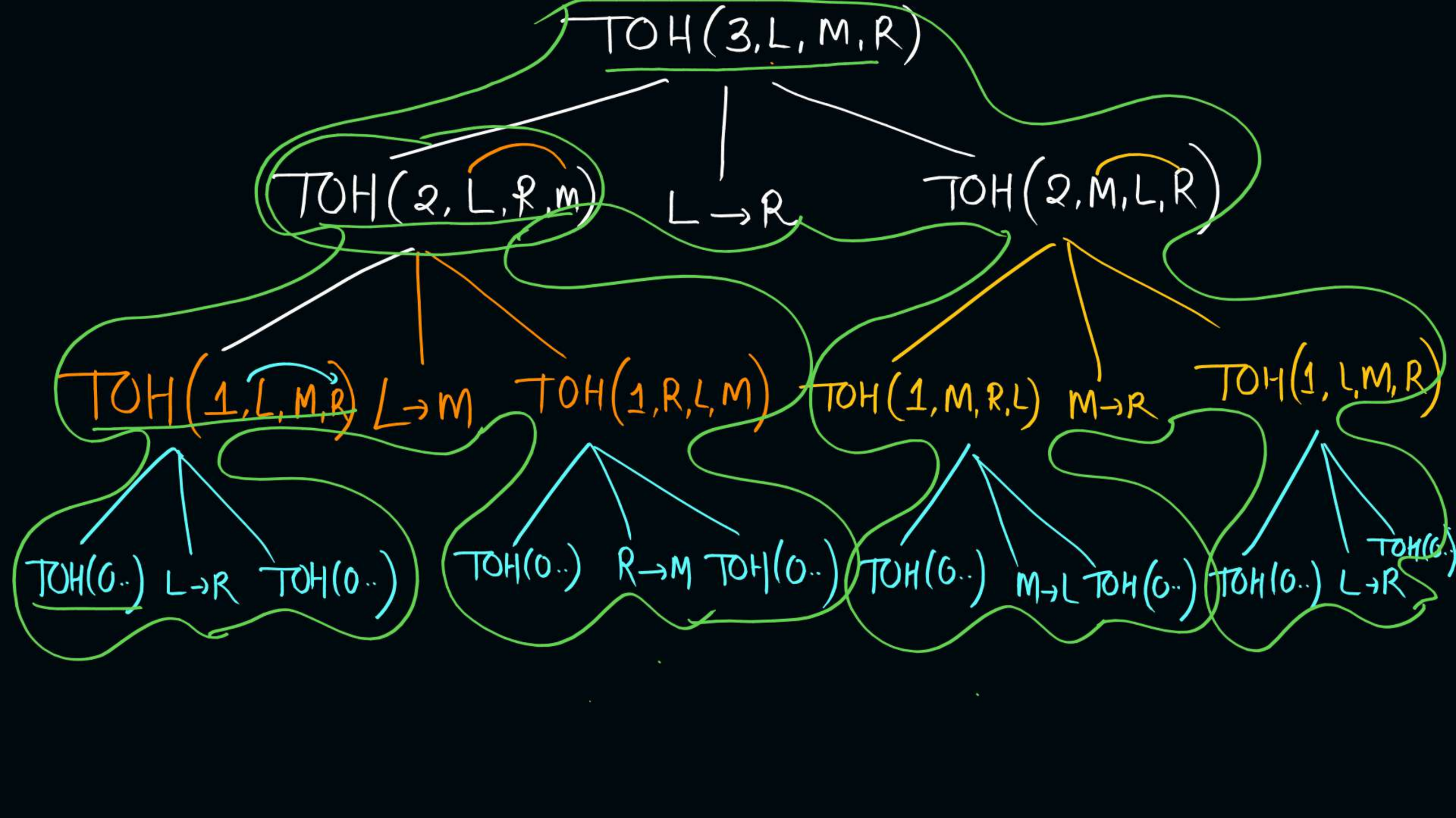
1 2 3

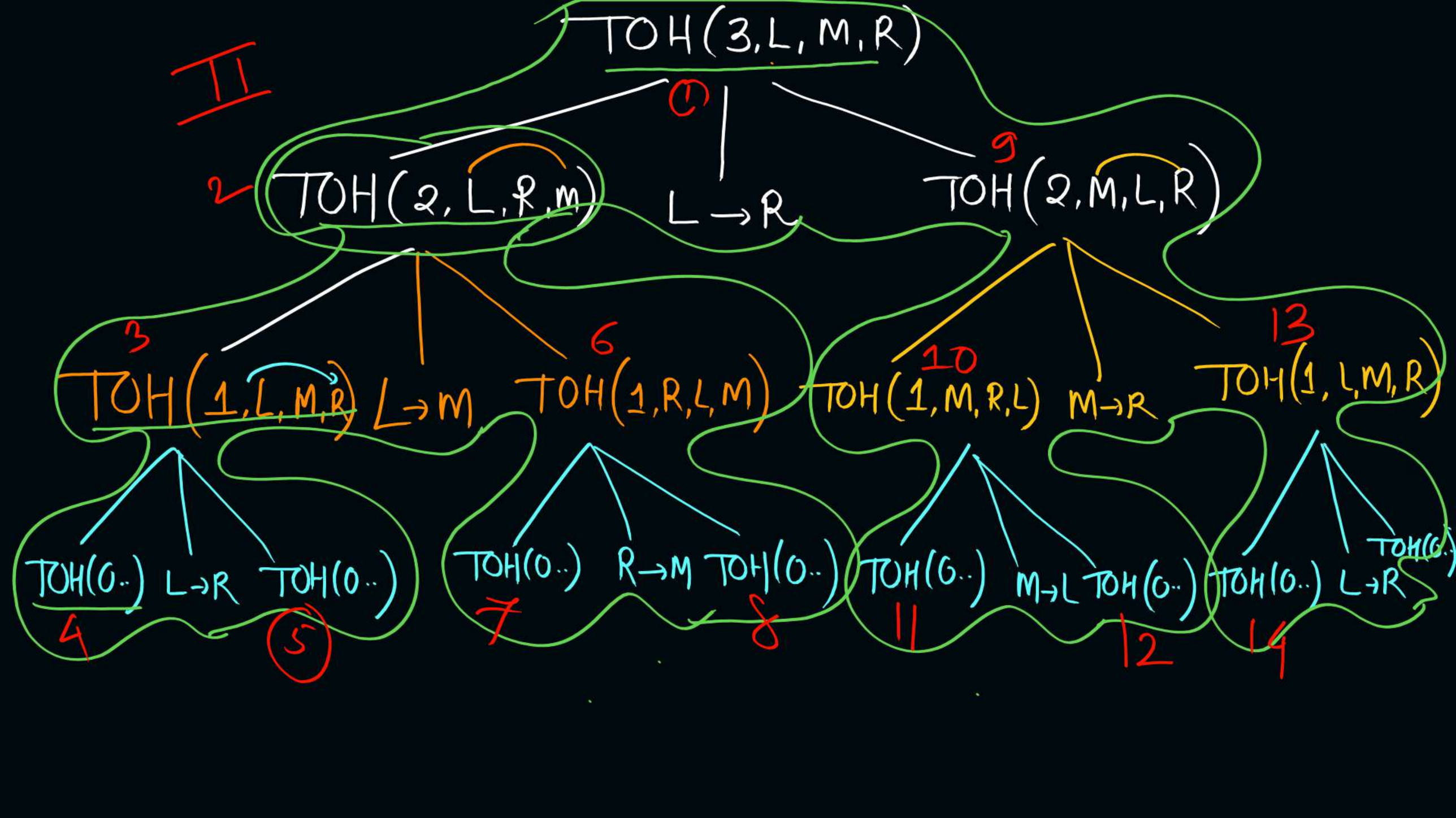
```
Algoorthm TOH( n, L, M, R) {
    if(n>=1) {
        TOH( n-1, L, R, M);
        Move the nth disc from L→R;
        TOH( n-1, M, L, R);
    }
}
```

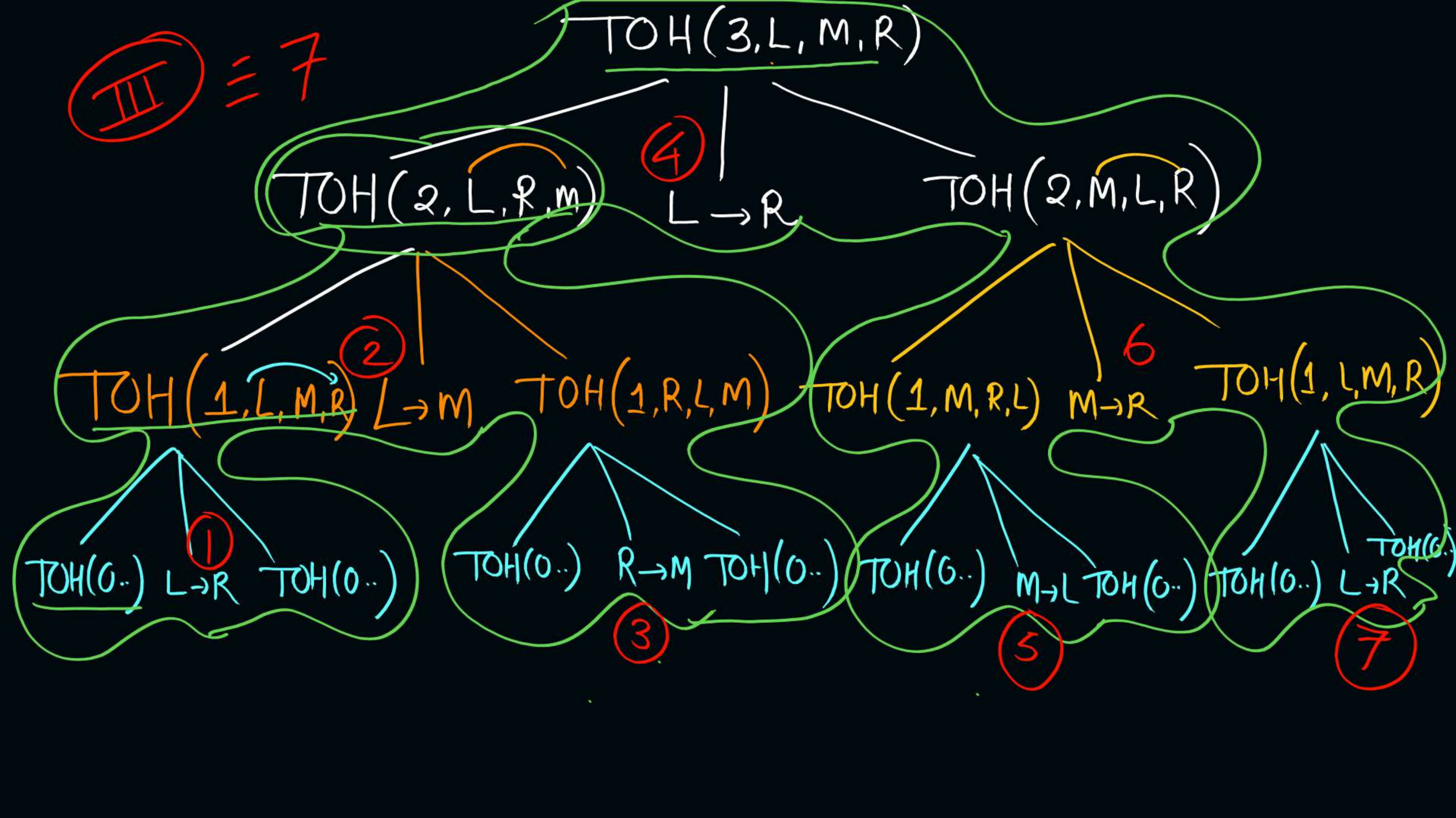
1 3 2

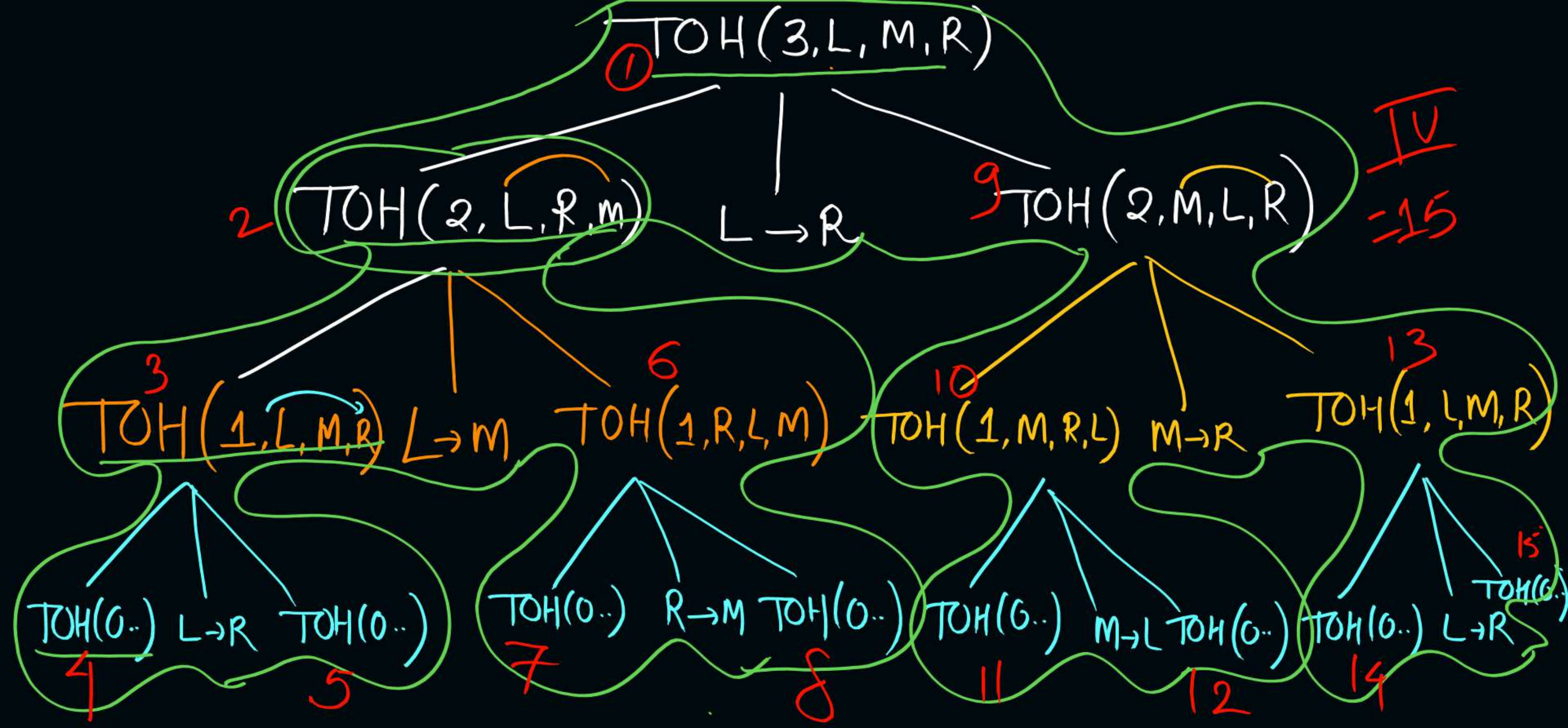
2 1 3

Slide


TOH(3,L,M,R)
TOH(2,L,R,M)
L→R
TOH(2,M,L,R)
TOH(1,L,M,R)
L→M
TOH(1,R,L,M)
TOH(1,M,R,L)
M→R
TOH(1,L,M,R)
TOH(0..)
L→R
TOH(0..)
TOH(0..)
R→M
TOH(0..)
TOH(0..)
M→L
TOH(0..)
TOH(0..)
L→R
TOH(0..)


II
TOH(3,L,M,R)
1
2
TOH(2,L,R,M)
L→R
9
TOH(2,M,L,R)
3
TOH(1,L,M,R)
L→M
6
TOH(1,R,L,M)
10
TOH(1,M,R,L)
M→R
13
TOH(1,L,M,R)
TOH(0..)
L→R
TOH(0..)
4
5
TOH(0..)
R→M
TOH(0..)
7
8
TOH(0..)
M→L
TOH(0..)
11
12
TOH(0..)
L→R
TOH(0..)
14


III = 7
TOH(3,L,M,R)
TOH(2,L,R,M)
4
L→R
TOH(2,M,L,R)
TOH(1,L,M,R)
2
L→M
TOH(1,R,L,M)
TOH(1,M,R,L)
M→R
6
TOH(1,L,M,R)
TOH(0..)
1
L→R
TOH(0..)
TOH(0..)
R→M
TOH(0..)
3
TOH(0..)
M→L
TOH(0..)
5
TOH(0..)
L→R
TOH(0..)
7


TOH(3,L,M,R)
1
TOH(2,L,R,M)
2
L→R
TOH(2,M,L,R)
9
IV
=15
TOH(1,L,M,R)
3
L→M
TOH(1,R,L,M)
6
TOH(1,M,R,L)
10
M→R
TOH(1,L,M,R)
13
TOH(0..)
4
L→R
TOH(0..)
5
TOH(0..)
7
R→M
TOH(0..)
8
TOH(0..)
11
M→L
TOH(0..)
12
TOH(0..)
14
L→R
TOH(0..)
15



Tower of Hanoi

10. If the number of disks are 3 in **Towers of Hanoi**

(a) After how many Invocations, there is a First move 4

(b) After how many Invocations, there is a Last move 14

(c) Total moves 7

(d) Total Invocations 15

(A) Innvocation
function call



Tower of Hanoi

10. If the number of disks are n in **Towers of Hanoi**

(a) After how many Invocations, there is a First move n+1

(b) After how many Invocations, there is a Last move $2^{n+1} - 2$

(c) Total moves $2^n - 1$

(d) Total Invocations $2^{n+1} - 1$

(A) Innvocahon funchon call



Fibonacci series

```
int fib(int n)
{
    if (n <= 1)
    return n;

    return fib(n-1) + fib(n-2);
}
```

Draw the recursion tree for fib(5)

fib(0) = 0

fib(1) = 1

0, 1, 1, 2, 3, 5, 8. - - -

fib(2) fib(3)

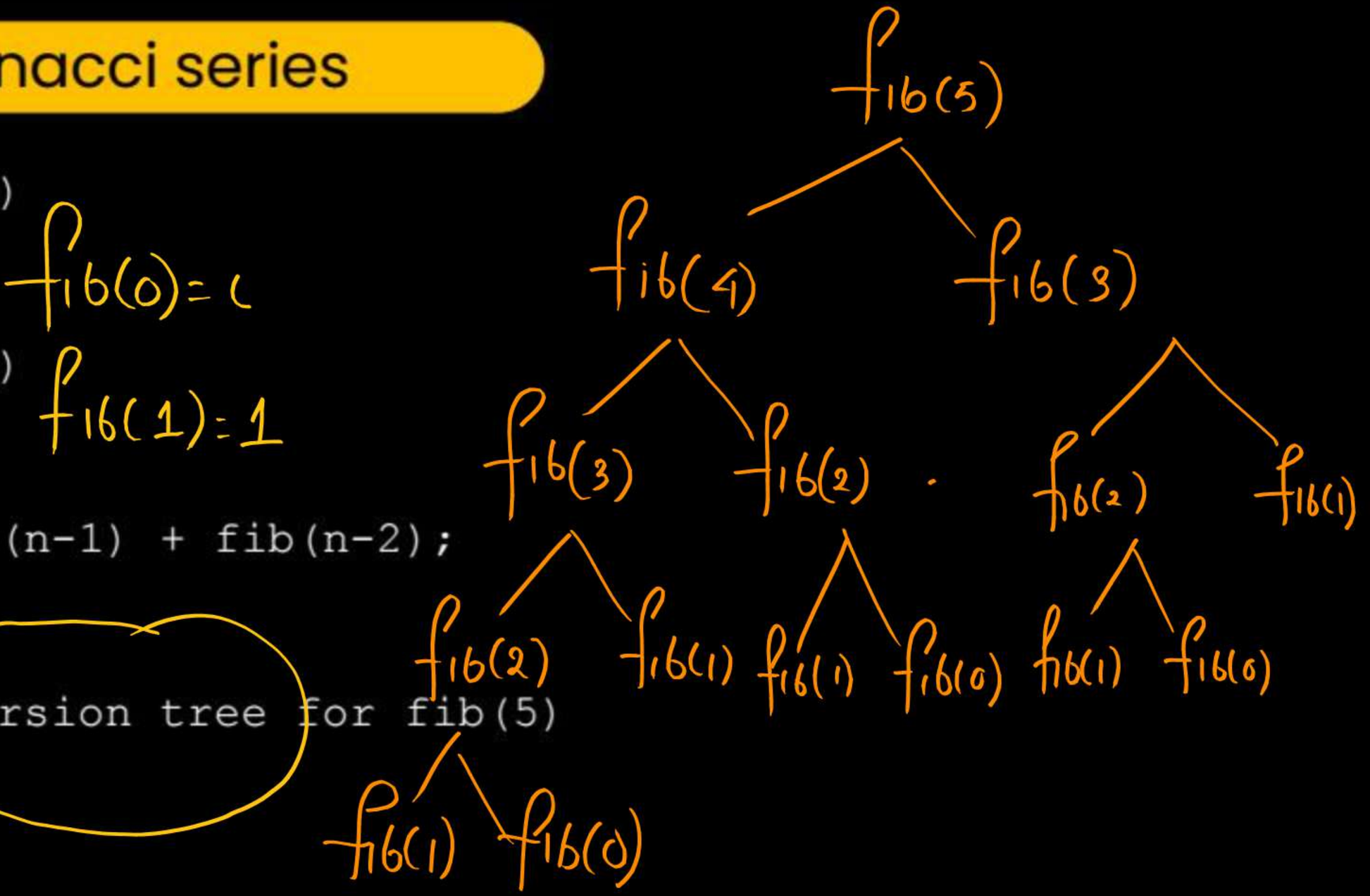


Fibonacci series

```
int fib(int n)
{
    if (n <= 1)
    return n;

    return fib(n-1) + fib(n-2);
}
```

Draw the recursion tree for fib(5)





Fibonacci series

```
int fib(int n)
{
    if (n <= 1)
    return n;
    return fib(n-1) + fib(n-2);
}
```

Calculate value of fib(8)

Dont draw Recursion tree

Value Calculation of Recursion function

$fib(0) = 0$

$fib(1) = 1$

$fib(2) = fib(0) + fib(1)$

$= 0 + 1 = 1$

$fib(3) = 2$

$fib(4) = 3$

$fib(5) = 5$

$fib(6) = 8$

$fib(7) = 13$

$fib(8) = 21$

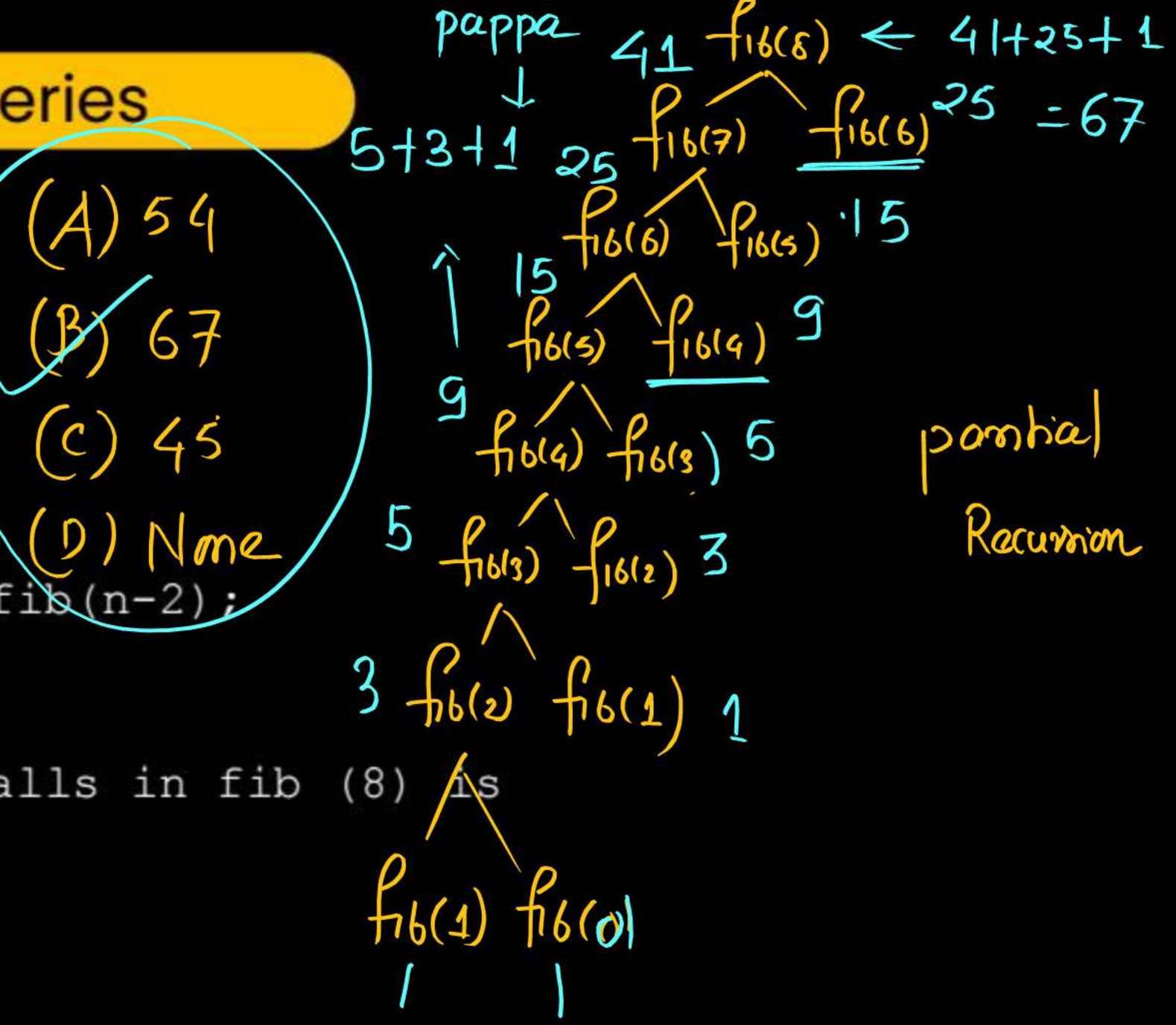


Fibonacci series

```
int fib(int n)
{
    if (n <= 1)
    return n;
    return fib(n-1) + fib(n-2);
}
```

The total number of calls in fib (8) is

(A) 54
(B) 67
(C) 45
(D) None





Fibonacci series

```
int fib(int n)
{
    if (n <= 1)
    return n;
    return fib(n-1) + fib(n-2);
}
```

The total no. of '+' in fib (9) is

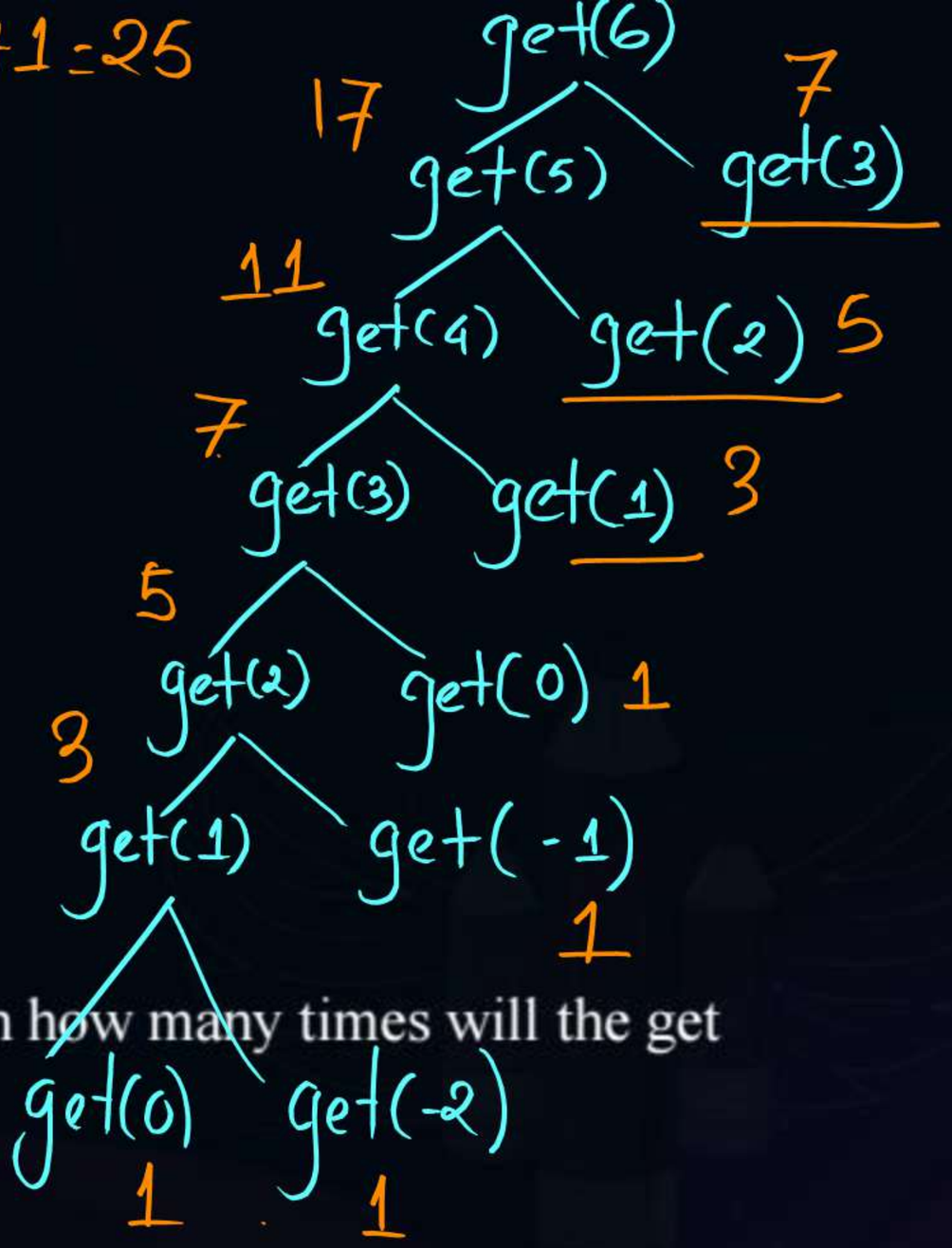
Question

Consider the following recursive C function.

```
void get(int n){
    if (n<1) return;
    get (n-1);
    get (n-3);
    printf("%d", n);
}
```

If function get(6) is being called in main() then how many times will the get function be invoked before returning to the ?

(A) 15 (B) 25 (C)35 (D) 45



Slide

Consider the C function given below

```
int f(int j){
static int i = 50;
int k;
if (i == j){
        printf("something");
        k = f(i);
        return 0;
}
else
        return 0;
```

j=50 - stack overflow ← No output

j=50 ← Something

Which one of the following is TRUE?

A) The function returns 0 for all values of j. ✗

B) The function prints the string something for all values of j. ✗

C) The function returns 0 when j = 50. ✗

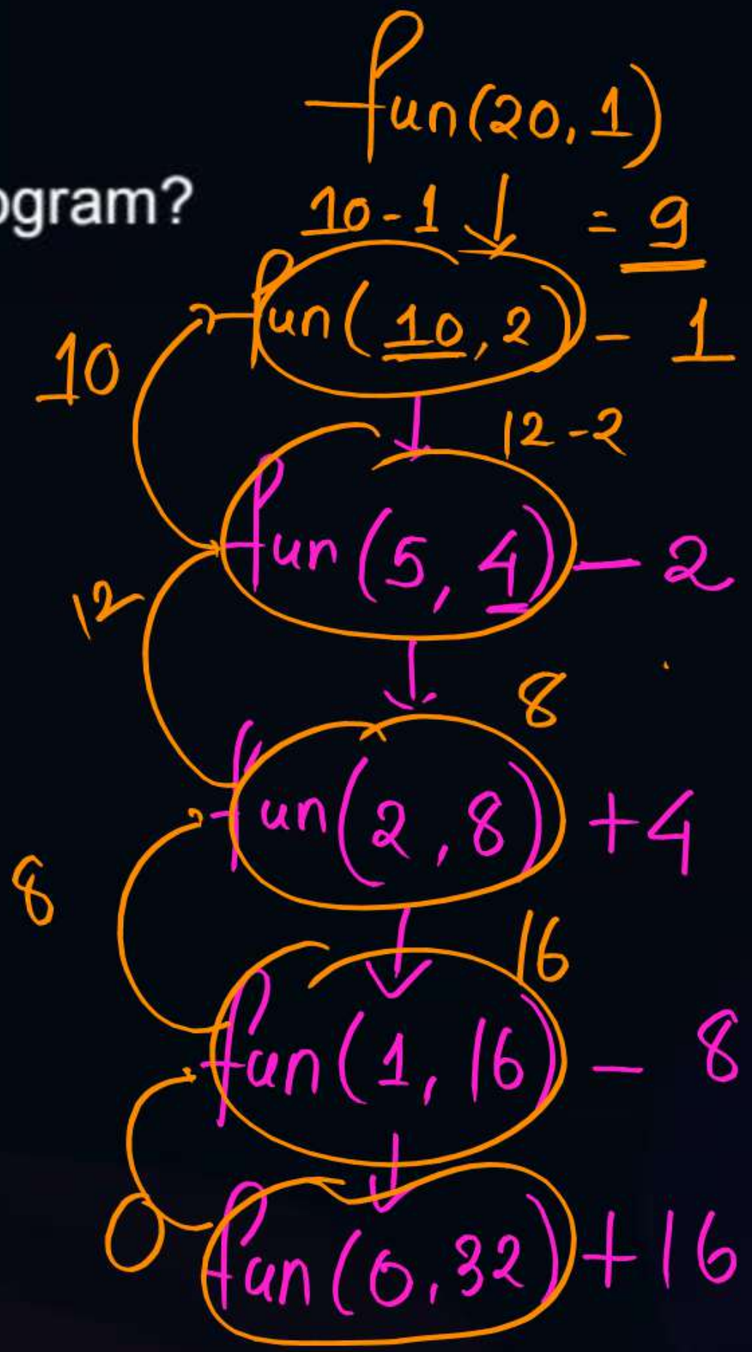
D) The function will exhaust the runtime stack or run into an infinite loop when j = 50.

Question

What is the output printed by the following program?

```
# include <stdio.h>
int f(int n, int k) {
if (n == 0) return 0;
else
if (n%2)
  return f(n/2, 2*k) + k;
else
  return f(n/2, 2*k) - k;
}
```

(A) 5
(B) 8
(C) 9
(D) 20





Ackermann function

$$A(x,y) = \begin{cases} y+1 & if\ x = 0 \\ A(x-1,1) & if\ y = 0 \\ A(x-1, A(x,y-1)) & otherwise \end{cases}$$

A(1,2) =

$A(1,2)$

$\downarrow$

$A(0, A(1,1))$

$\downarrow$

$A(0, A(0, A(1,0)))$

$\downarrow$

$A(0, A(0,1))$

$\downarrow$

111

Ackermann function

$$A(x,y) = \begin{cases} y+1 & if\ x = 0 \\ A(x-1,1) & if\ y = 0 \\ A(x-1, A(x,y-1)) & otherwise \end{cases}$$

A(1,2) =

$A(1,2)$

$A(0, A(1,1))$

$A(1,1)$

$A(0,2)$

$A(0,1)$

y+1

1+1

Ackermann function

$$A(x,y) = \begin{cases} y+1 & if\ x = 0 \\ A(x-1,1) & if\ y = 0 \\ A(x-1, A(x,y-1)) & otherwise \end{cases}$$

A(1,2) =

$A(1,2)$

$\downarrow$

$A(0, A(1,1))$

$\downarrow$

$A(0,1)$

$\downarrow$

$1+1$

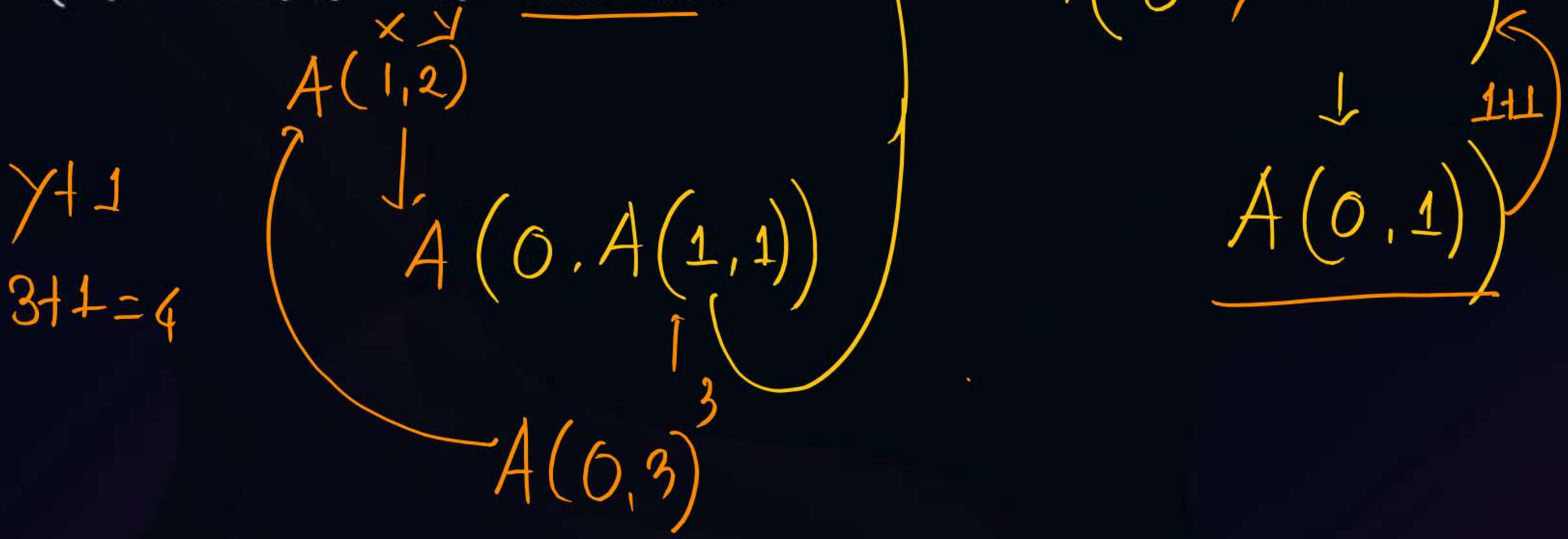
$2+1=3$

$y+1$

Ackermann function

$$A(x,y) = \begin{cases} y+1 & if\ x = 0 \\ A(x-1,1) & if\ y = 0 \\ A(x-1, A(x,y-1)) & otherwise \end{cases}$$

A(1,2) =





Question

#Q. Consider the following program

```
int fun(int x){
    if(x>3)
        return fun(x-4) + fun(x-1) + 1;
    return 1;
}
```

Find the value returned by fun(5)

fun(0) = 1

fun(1) = 1

fun(2) = 1

fun(3) = 1

fun(4) = fun(0) + fun(3) + 1
= 1 + 1 + 1 = 3

fun(5) = fun(1) + fun(4) + 1

1 + 3 + 1 = 5

Slide



2 mins Summary

Topic — TOH

Topic — Counting function call

Topic — Computation of value.

Topic

Topic

Slide

THANK - YOU

